



UNIVERSIDADE D
COIMBRA

Manual do Programador

Princípios de Programação Procedimental

Licenciatura em Engenharia informática
(1º ano)

Princípios de Programação Procedimental

Ano letivo: 2021/22

Docente: Maria José Patrício Marcelino

Índice

1. Introdução	3
2. Estrutura Geral do Programa	4
2.1. Estruturas	5
2.2. Main ()	6
3. Funcionalidades do programa	7
3.1. Funções	7
3.2. Função extra	8
4. Ficheiros	9
4.1. Ficheiro “dados.txt”	9
4.2. Ficheiro “dados.txt”	10

1. Introdução

O seguinte manual do programador pretende descrever as funcionalidades do programa desenvolvido no âmbito da cadeira do 2º semestre, do 1º ano de Licenciatura de Engenharia Informática – “Princípios de Programação Procedimental”.

Este projeto foi desenvolvido no aplicativo CLion em linguagem C e tem como objetivo a construção de uma aplicação capaz de gerir as contas de saldo escolar dos alunos de uma escola de ensino secundário. A fim de alcançar o objetivo proposto, o programa é constituído por três partes essenciais: uma função principal *main()*, uma *source file* que é composto pelas funções e a *header file* com as estruturas e definições das funções que são usadas ao longo do *main()* e da *source file*.

Foram cumpridos todos os requisitos impostos para a realização do programa salvaguardando a eficácia do ponto de vista de memória ocupada, sendo ainda adicionada a função (“transferir_saldo”) que, como diz o nome, permite transferir saldo entre os alunos na plataforma usada.

Toda a informação relevante sobre os alunos deve ser armazenada em ficheiros de texto, os quais devem ser carregados quando o programa é iniciado e podem ser atualizados sempre que necessário de modo que não haja perda de informação. O programa encontra-se também devidamente protegido.

Com o intuito de cumprir o solicitado, o manual do programador descreve o modo como o programa foi implementado, facilitando a compreensão do código fonte.

2. Estrutura Geral do Programa

A implementação das funcionalidades do programa encontra-se distribuída por duas *source file* .c:

- “main.c” – corpo principal do programa, onde são chamadas as funções desenvolvidas em “Source.File.c”;
- “Source.File.c” – composto por todas as funções desenvolvidas exeto a principal;

Com visão de facilitar a leitura da estrutura geral da *header file*, à qual denominou-se de “header_file.h”. Esta é formada por estruturas de dados utilizadas ao longo do programa, assim como, os cabeçalhos de todas as funções desenvolvidas:

```
//Lista_alunos:
pLista_aluno cria();
void insere(pLista_aluno lista, aluno a1);
void procura (pLista_aluno lista, int numero, pLista_aluno *ant, pLista_aluno *atual);

// Lista de despesas
pLista_despesas cria_d();
void insere_d (pLista_despesas lista,despesa d1);

//Funcoes:
int str_i(const char *d);
int verifica_existencia(int num, pLista_aluno lista);
aluno inserir_dados_aluno();
void elimina (pLista_aluno lista, int chave);
void listagem_alfabetica(pLista_aluno lista);
void listar_saldo(pLista_aluno lista, double valor);
aluno imprimir_aluno(pLista_aluno lista, pLista_despesas lista1, int num);
aluno efetuar_despesa(pLista_despesas lista, pLista_aluno listaA, int num, double valor);
aluno carregar_saldo(pLista_aluno lista, double valor);
aluno efetuar_transferencia(pLista_despesas lista, pLista_aluno listaA, int num, double valor);
aluno transferir(pLista_aluno lista,int num, double valor);
aluno transferir_saldo(pLista_aluno lista,pLista_despesas listaD,int num1,int num2,double valor);
void guardar(FILE *f, pLista_aluno lista);
void guardar_d(FILE *f,pLista_despesas lista);
```

Figura 1

2.1 Estruturas

Relativamente às estruturas usadas no decorrer do código, têm como cargo a simplificação e maior fluidez no desenvolvimento da aplicação.

Assim, foram criadas as seguintes estruturas:

```
typedef struct data{  
    int dia;  
    int mes;  
    int ano;  
}data;
```

Figura 2

```
typedef struct despesa{  
    int numero;  
    double valor;  
    char descricao[50];  
    data data1;  
}despesa;
```

Figura 3

```
typedef struct aluno{  
    char nome[50];  
    data data_nascimento;  
    int ano;  
    char turma[10];  
    int numero;  
    double saldo;  
} aluno;
```

Figura 4

```
typedef struct noLista_despesas{  
    despesa despesalista;  
    struct noLista_despesas * prox;  
} noLista_despesas;
```

```
typedef noLista_despesas *pLista_despesas;
```

Figura 5

A estrutura “despesa” contém a as informações necessárias na descrição de uma despesa: o numero do aluno ao qual efetuou a despesa, o valor, descrição e data;

A estrutura “aluno” corresponde a todas as informações de um determinado aluno, isto é, o nome, a sua data de nascimento, o ano que frequenta, a turma, o seu respetivo número e o saldo disponível.

A struct correspondente à figura 5 e figura 6 correspondem às listas ligadas das despesas e dos alunos, respetivamente.

```
typedef struct noLista_alunos{
    aluno alunolista;
    struct noLista_alunos *prox;
} noLista_alunos;
typedef noLista_alunos * pLista_alunos;
```

Figura 6

2.2 Main ()

O *main ()* refere-se, como foi transmitido anteriormente, à função principal do código, isto é, a base da execução do programa.

Ao inicializar o programa, este percorre os ficheiros que contém a base de dados do programa, onde são armazenados os registos dos alunos e as suas respetivas despesas, e executa a leitura destes como é possível observar até à linha 93. Estes dois ficheiros são separados em lista de alunos e lista de despesas, correspondentes aos ficheiros “dados.txt” e “despesas.txt”, respetivamente, permitindo uma melhor interpretação do código.

Após o processo de leitura, é efetuado um ciclo onde será pedido ao usuário a funcionalidade que este deseja realizar. Este ciclo é feito através de um *switch* para uma melhor estruturação do código, que apenas é encerrado quando é inserido “0” no interpretador.

Este *switch* é composto por oito opções correspondentes às sete funções requeridas e à função extra implementada no programa.

No decorrer do código, no final do código, assim como, no final de cada operação que executa qualquer tipo de alteração nos dados existentes (p.e.: inserir um novo aluno; eliminar um aluno existente; etc.), é guardado automaticamente as alterações da mesma, permitindo, assim, a diminuição do risco de perda de dados.

3. Funcionalidades

Relativamente às funções, como nos foi pedido, desenvolvemos um programa acompanhado com sete funções distintas:

1. Inserir um novo aluno;
2. Eliminar um aluno existente;
3. Listar todos os alunos por ordem alfabética;
4. Listar os alunos com saldo abaixo de um determinado valor (por ordem decrescente de saldos);
5. Apresentar toda a informação de um determinado aluno;
6. Efetuar uma despesa por um determinado aluno;
7. Carregar o saldo de um aluno com um valor.

3.1 Funções

Sobre as funções referidas anteriormente, iremos explicar um pouco o constituinte de algumas destas:

Listagem_alfabética:

Esta função lista todos os alunos por ordem alfabética. É criada uma variável local “contador”, que conta o número total de alunos guardados na lista de alunos.

Depois é criada uma matriz de strings onde são adicionados os nomes dos alunos.

```
char nomes[contador][50];
for (int i = 0; i < contador; ++i) {
    auxlista = auxlista->prox;
    strcpy( Dest: nomes[i], Source: auxlista->alunoLista.nome);
}
char aux[50];
for (int i = 0; i < contador-1; ++i) {
    for (int j = i+1; j < contador; ++j) {
        if(strcmp(nomes[i],nomes[j]) > 0){
            strcpy( Dest: aux, Source: nomes[i]);
            strcpy( Dest: nomes[i], Source: nomes[j]);
            strcpy( Dest: nomes[j], Source: aux);
        }
    }
}
```

Figura 7

Por fim, os nomes são ordenados dentro da matriz com o auxílio da função “strcpy()” e imprimidos no interpretador.

Listar_saldo:

A função imprime o nome de todos os alunos com saldo inferior ao atribuído pelo utilizador ao parâmetro da função “valor” por ordem decrescente.

É criada uma variável local contador, que conta o número total de alunos com saldo inferior ao selecionado.

Posteriormente, inicia-se um ciclo que é repetido o número de vezes guardado na variável contador, sendo que, em cada um, procura e imprime o aluno com maior saldo inferior ao valor selecionado.

```
} while (auxlista1 != NULL);
contador = contador-1;
if(auxlista2 != NULL) {
    do {
        if (valor != auxlista2->alunoLista.saldo && aux > auxlista2->alunoLista.saldo && aux1 < auxlista2->alunoLista.saldo) {
            aux1 = auxlista2->alunoLista.saldo;
            auxlista3 = auxlista2;
        }
        auxlista2 = auxlista2->prox;
        if (contador != 0 && auxlista2 == NULL) {
            aux = aux1;
            printf( format: "-- %s --", auxlista3->alunoLista.nome);
            aux1 = 0;
            auxlista2 = lista->prox;
            contador = contador - 1;
        }
    } while (contador != 0);
}
```

Figura 8

Nota: O valor selecionado vai sendo atualizado com o valor do saldo do aluno imprimido no ciclo anterior.

3.2 Função Extra

A funcionalidade extra do nosso programa tem como função transferir saldo entre alunos e usa como recurso outras duas funções para uma melhor interpretação da mesma.

A função “efetuar_transferência” retira o dinheiro ao aluno que realiza a transferência e adiciona a respetiva transferência à Lista de despesas.

A função “transferir” adiciona o valor transferido ao saldo do destinatário.

4. Ficheiros:

A fim de guardar/armazenar as informações sobre os alunos e as suas despesas quando o programa é encerrado é utilizado o recurso a dois ficheiros, como foi referido anteriormente:

- “dados.txt”;
- “despesas.txt”

4.1 Ficheiro “dados.txt”:

Neste ficheiro estão contidas todas as informações dos alunos que são guardadas com o auxílio da “*struct* aluno”.

A organização da informação neste ficheiro é feita da seguinte forma:

- A informação de cada aluno é delimitada por uma linha vazia (através do uso do caracter, ‘\n’);
- Os dados de cada aluno estão organizados linha a linha pela seguinte ordem: nome, data de nascimento, ano, turma, número, saldo.
- A informação deste esta organizada pelos números dos alunos por ordem crescente.

```
Nome: Emanuel Ferreira
Data de nascimento: 11/2/2004
Ano: 12
Turma: A
Número: 1
Saldo: 10.000000
```

4.2 Ficheiro “despesas.txt”:

Este ficheiro armazena apenas a informação sobre as despesas dos alunos com o auxílio da “*struct* despesa”.

A organização da informação neste ficheiro é feita da seguinte forma:

- A informação de cada aluno é separada por uma linha vazia (através do uso do carácter, ‘\n’);
- Os dados sobre as despesas de cada aluno estão organizados linha a linha pela seguinte ordem: número, valor, descrição, data.
- A informação deste está organizada pelos números dos alunos por ordem crescente.

```
Número: 1
Valor: 3.000000
Descrição: Gelado
Data: 18/6/2022
```